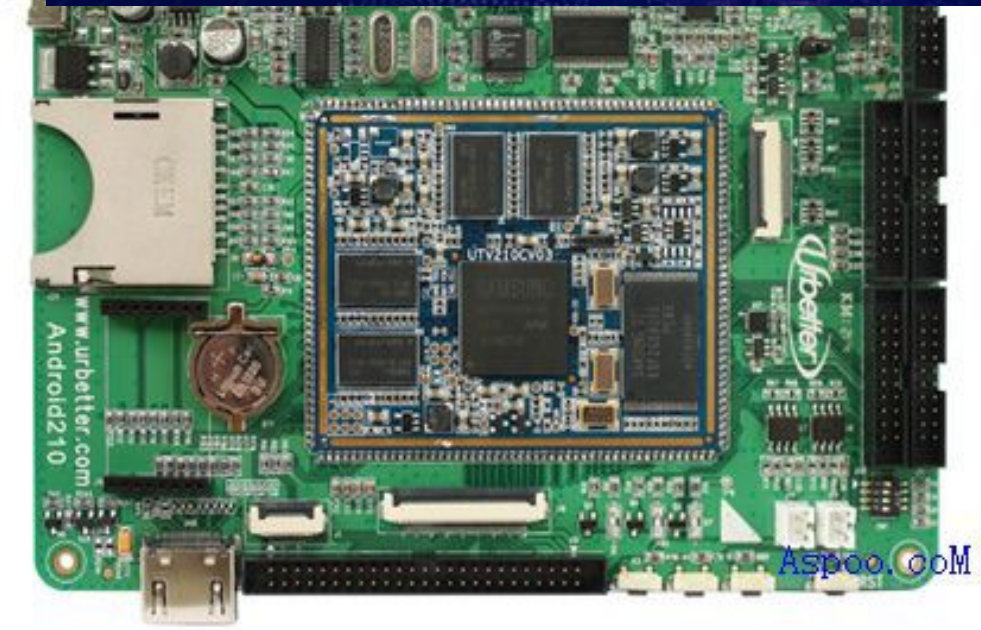
第4章

中科(厦门)数据智能研究院
XIAMEN INSTITUTE OF DATA INTELLIGENCE
厦门市开源芯片产业促进会
XIAMEN RISC-V INDUSTRY ASSOCIATION
RISC-V AI指令集的
标准化与开源实现
3月26日 21：00-22：00


www.urbetter.com
Android210
Urbetter
Aspoo.coM

目　录

4.1 基于S5PV210微处理器的硬件平台体系结构
4.2 存储系统
4.3 时钟系统
4.4 GPIO接口
4.5 串行通信接口
4.6 A/D转换器
4.7 本章小结

S5PV210又名“蜂鸟”，是三星推出的一款适用于智能手机和平板电脑等多媒体设备的应用处理器。

S5PV210采用了ARM Cortex-A8内核，ARM V7指令集，主频可达1GHZ，64/32位内部总线结构，32/32KB的数据/指令一级缓存，512KB的二级缓存，可以实现2000DMIPS（每秒运算2亿条指令集）的高性能运算能力。该处理器同时也提供了丰富的外设，以应用于各种嵌入式系统设计之中。

本章将简要介绍S5PV210处理器的硬件结构，主要外设的功能及其接口。

4.1 Part One

基于S5PV210微处理器的硬件平台体系结构

4.1.1 S5PV210处理器简介

S5PV210是一款高效率、高性能、低功耗的32位RISC处理器，它集成了 ARM Cortex-A8核心，实现了ARM架构V7并且支持众多外围设备。

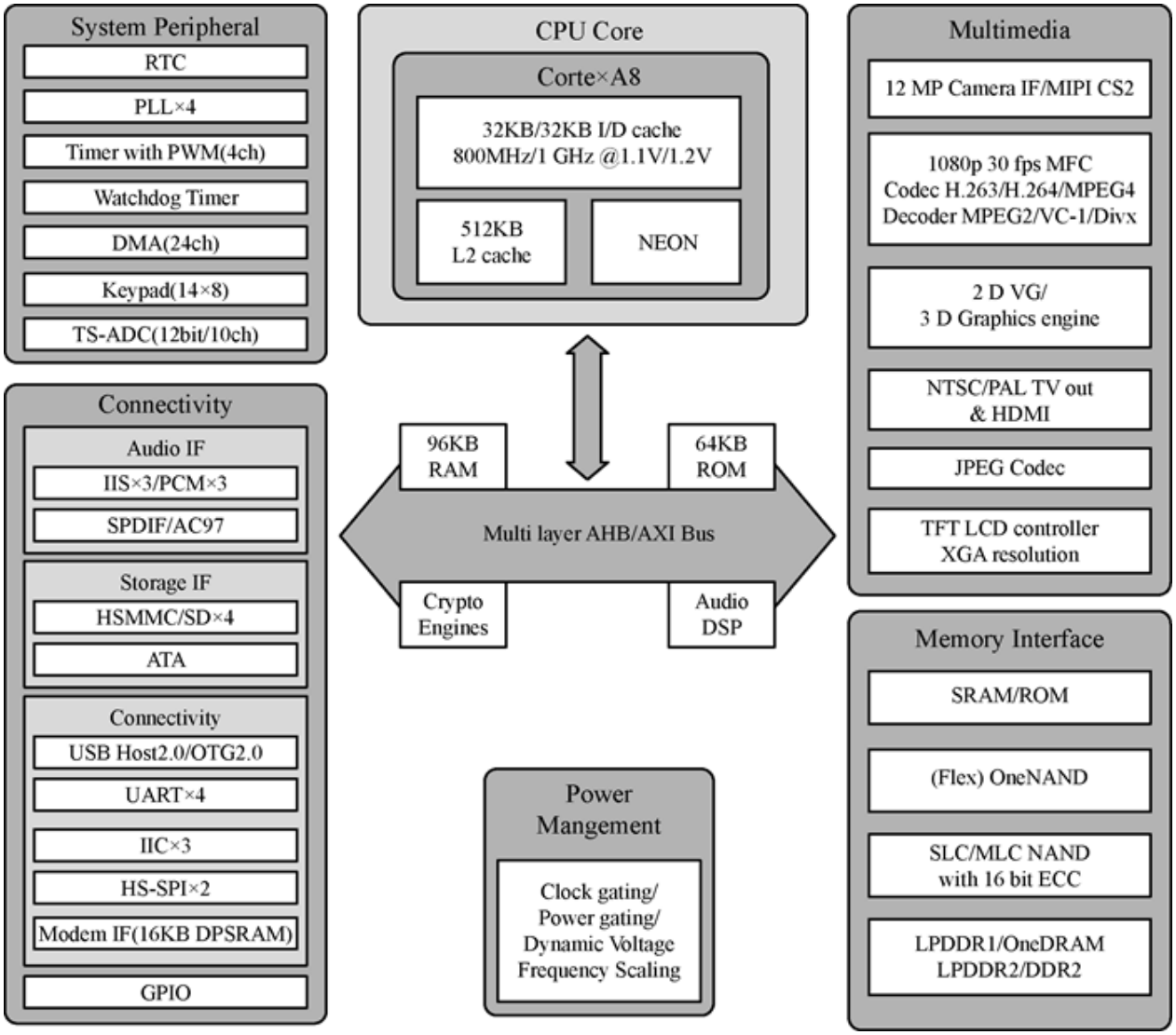
S5PV210采用64位内部总线结构，为3G和3.5G通信服务保证最优化的硬件性能，并且提供了许多强大的硬件加速器，例如运动视频处理、显示控制及缩放等。

S5PV210具有多种外部存储器接口，能够承受大内存在高端通信服务所需的带宽，例如其DRAM控制器支持LPDDR1、DDR2或LPDDR2的存储器扩展，其FLASH/ROM接口支持NAND闪存、NOR闪存、OneNAND闪存、SRAM和ROM类型的外部存储器。

为了降低系统的总成本并且提高整体功能，S5PV210微处理器内部集成了众多外设。

如TFT真彩LCD控制器、摄像头接口、MIPI DSI显示串行接口、电源管理、ATA接口、4个通用异步收发器、24通道的DMA、4个定时器、通用I/O端口、3个I2S、IIC接口、两个HS-SPI、USB Host2.0、高速运行的USB2.0 OTG、4个SD Host和高速多媒体接口

S5PV210处理器主要由6大部分组成，分别为**CPU核心**、**系统外设**、**多媒体**、**电源管理**、***存储器接口***和**Connectivity模块**。CPU和各个部分之间通过多层次AHB/AXI总线进行通信。



4.1.2 S5PV210内部各模块介绍

1. CPU内核

（1）Cortex A8处理器

运行速度在600MHZ～1GHZ时，Cortex A8处理器符合功率优化的移动设备小于300mW状态下运行的要求，同时符合性能优化的消费类应用需要2000Dhrystone MIPS的要求。支持第一个超标量处理器，用于增强代码密度和性能。支持JazelleRCT技术用于超前和即时编译的Java和其他字节语言。13级主整数流水线。

（2）NEON：

CortexA8处理器内部集成的可以实现复杂算法的模块，比如图像的智能分析、数学上的运算等可以通过NEON来实现。

（3）32KB I/O缓存、512KB L2 Cache。

2．系统外设

（1）RTC实时时钟

提供完整的时钟功能：秒、分、小时、日、月、年。使用32.768KHZ时钟基准。提供报警中断。提供定时器时钟节拍中断。

（2）PLL锁相环

芯片具有4个锁相环(PLL)，分别为APLL/MPL/EPLL/VPLL。APLL产生ARM核心和MSYS时钟。EPLL生成特殊的时钟。VPLL为视频接口生成时钟。

（3）具有脉宽调制功能的定时器

具有4通道32位内部定时器，3通道带脉宽调制功能以及可编程工作周期、频率和极性。具有死区产生功能。支持外部时钟源。

（4）看门狗定时器——16位看门狗定时器。

（5）DMA

具有特定的指令集提供DMA传输的灵活性。内置增强型8通道的DMA。内存到内存转换DMA多达16组，外设到内存转换DMA支持多达8组。

（6）Keypad

支持14×8矩阵键盘。提供内部消抖功能。

（7）ADC转换器

10通道多路复用ADC。支持最大500K采样率和12位的分辨率。

3. 多媒体

（1）摄像头接口

支持多输入包括ITU-R BT601/656模式、DMA模式和MIPI模式。支持多输出包括DMA模式和直接FIFO模式。支持数码变焦功能。支持图像镜像和旋转功能。支持生成各种图像格式。支持捕捉画面管理。支持图像效果。

（2）多格式视频编解码器

ITU-TH.264、ISO/IEC 14496-10编解码支持基线/主/High Profile的4.0级，编码支持基线/主/高属性。ITU-TH.263 Profile level3编解码支持Profile3，限制SD分辨率每秒30帧，支持基线配置文件的编码。编码支持MPEG-4简单类/高级简单类。ISO/IEC 13818-2 MPEG-2编解码支持主要的轮廓高度，解码支持MPEG-1。

（3）JPEG编码器

支持压缩/解压到65536x65536分辨率。支持的压缩格式即输入原始图像为YCbCr422或RGB565，输出JPEG文件为基线JPEG格式的YCbCr422或YCbCr420。支持的解压缩格式即输入JPEG文件为基线YCbCr444或YCbCr420或YCbCr422格式、JPEG或灰色，输出原始图像的YCbCr422或YCbCr420格式。支持通用的色彩空间转换器。

（4）3D图形引擎

支持3D图形、矢量图形、视频编码和解码。具有通用可扩展渲染引擎、多线程引擎和顶点着色器功能。支持8000x8000的图像尺寸。支持90/180/270度旋转。支持16/24/35bpp，24位颜色格式。

（5）模拟电视接口

输出视频格式为NTSC/PAL。支持的输入格式即ITU-R BT.601的YCbCr444。支持480i/p和576i协议。支持复合视频。

（6）液晶显示器接口

支持24/18/16 bpp的并行RGB接口的LCD。支持8/6bpp串行RGB接口。支持双i80接口的LCD。支持典型的屏幕尺寸：1024x768、800x480、640x480、320x240和160x160。虚拟图像达到16M像素。ITU-BT601/656格式输出。

4．电源管理

（1）时钟门控功能。

（2）各种低功耗模式可供选择，如空闲、停止、深度空闲和睡眠模式。

（3）睡眠模式下唤醒源可以是外部中断、RTC报警、计时器节拍。

（4）停止和深度空闲模式唤醒源可以是触摸屏人机界面、系统定时器等。

5．存储器接口

（1）SRAM/ROM/NOR接口

8位或16位的数据总线。地址范围支持23位。支持异步接口。支持字节和半字访问。

（2）OneNAND闪存接口

16位的数据总线。地址范围支持16位。支持字节和半字访问。Flex OneNAND闪存支持2KB页面模式，OneNAND闪存支持4KB页面模式。支持专用的DMA。

（3）NAND接口

支持行业标准的NAND接口。8位的数据总线。

（4）LPDDR1接口

32位数据总线，1.8V接口电压。每端口密度支持高达4GB（2CS）。

（5）DDR2接口

32位数据总线将支持400Mbps/引脚双数据速率。1.8V接口电压。每端口密度支持高达1GB（2CS，4BANK的DDR2）。每端口密度支持高达4GB（1CS，8BANK的DDR2）。

6．Connectivity模块

（1）音频接口

AC97音频接口：

- 独立通道的立体声PCM输入、立体声PCM输出和单声道麦克风输入。
- 16位立体声音频。
- 可变采样率AC97编解码器接口。
- 支持AC97规格。

PCM音频接口：

- 16位单声道音频接口。
- 仅工作在主控模式。
- 支持三种PCM端口。

IIS总线接口：

- 基于DMA操作的三个I2S总线音频编解码器接口。
- 串行8位、16位、24位每通道的数据传输。
- 支持I2S、MSB、LSB对齐的数据格式。
- 支持PCM5.1声道。
- 支持不同比特时钟频率和编解码器的时钟频率。
- 支持一个5.1通道I2S的端口和两个2通道I2S端口。

SPDIF接口：

- 线性PCM每个样本支持多达24位。
- 支持非线性PCM格式如AC3，MPEG1、MPEG2。
- 2x24位缓冲器交替地用数据填充。

（2）存储接口

HS-MMC/SDIO接口：

- 兼容4.0多媒体卡协议版本（HS-MMC）。
- 兼容2.0版本SD卡存储卡协议。
- 基于128KBFIFO的TX/RX。
- 4个HS-MMC端口或4个SDIO端口。

ATA控制器支持ATA/ATAPI-6接口。

（3）通用接口

USB2.0 OTG：

- 符合USB2.0 OTG 1.0a版本。
- 支持高达480Mbps的传输速度。
- 具有USB芯片收发器。

UART：

- 具有基于DMA和中断功能的4个UART。
- 支持5位、6位、7位、8位的串行数据发送和接收。
- 独立的256字节FIFO的UART0，64字节FIFO的UART1和16字节FIFO的UART2/3。
- 可编程的传输速率。
- 支持IrDA1.0 SIR模式。
- 支持回环模式测试。

I2C总线接口：

- 3个多主控I2C总线。
- 8位串行面向比特的双向数据传输，在标准模式下可以达到100Kbps。
- 快速模式下高达400Kbps。

SPI接口：

- 3个符合2.11版本串行外设接口协议的接口。
- 独立的64K字节FIFO的SPI0和16字节FIFO的SPI1。
- 支持基于DMA和中断操作。

GPIO接口：

- 237个多功能输入/输出端口。
- 支持178个外部中断。

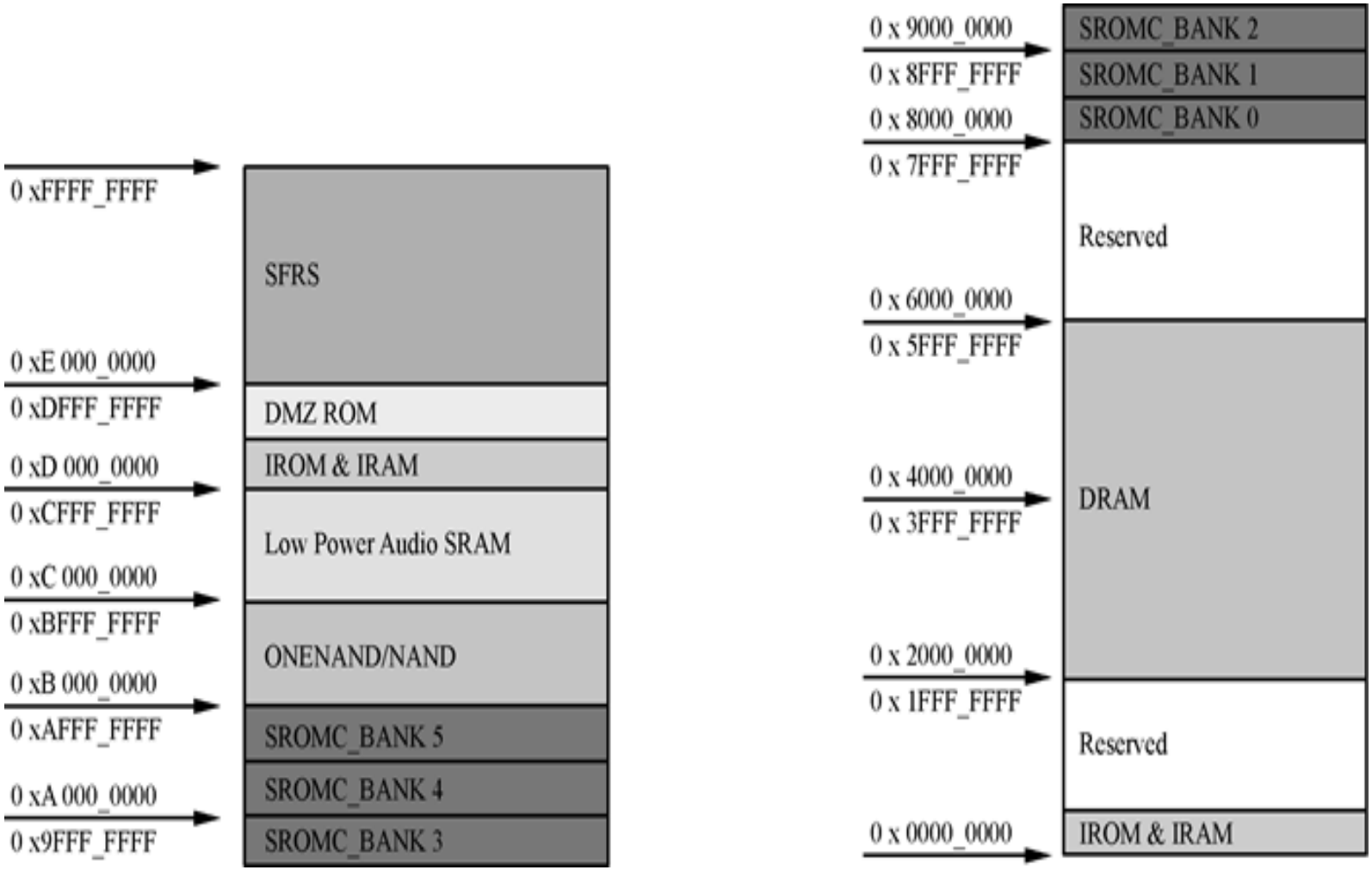
DC IN
POWER MANAGEMENT
BUZZER
SPEAKER
MATRIX
TRAFFIC LIGHT
WIFI+BT
ETHERNET
SEG
TEMP&HUMI
INFRARED
LIGHT SENSOR
DC MOTOR
STEPPER
RELAYS
KNOB
KEYPAD
KEYBOARD

4.2 Part Two

存储系统

4.2.1 S5PV210的地址空间

S5PV210的存储器地址映射如图4-2所示。



S5PV210的存储器地址空间分为7各部分，从下往上分别是引导区、动态随机存储器（DRAM）区、静态只读存储器（SROM）区、Flash区、音频存储区、隔离ROM区和特殊功能寄存器区。

地址		大小	描述	备注
0x0000_0000	0x1FFF_FFFF	512MB	Boot area	系统启动配置区，此映射区域由启动模式决定
0x2000_0000	0x3FFF_FFFF	512MB	DRAM 0	存配置区，用于DDR2 SDRAM寻址
0x4000_0000	0x5FFF_FFFF	512MB	DRAM 1	
0x8000_0000	0x87FF_FFFF	128MB	SROM Bank 0	外接总线型设备寻址区
0x8800_0000	0x8FFF_FFFF	128MB	SROM Bank 1	
0x9000_0000	0x97FF_FFFF	128MB	SROM Bank 2	
0x9800_0000	0x9FFF_FFFF	128MB	SROM Bank 3	
0xA000_0000	0xA7FF_FFFF	128MB	SROM Bank 4	
0xA800_0000	0xAFFF_FFFF	128MB	SROM Bank 5	
0xB000_0000	0xBFFF_FFFF	256MB	OneNAND/NAND Controller and SFR	OneNAND和NAND寻址区
0xC000_0000	0xCFFF_FFFF	256MB	MP3_SRAM output buffer	MP3_SROM输出缓存区
0xD000_0000	0xD000_FFFF	64KB	IROM	IROM区，设备引导使用
0xD001_0000	0xD001_FFFF	64KB	Reserved	保留区
0xD002_0000	0xD003_7FFF	96KB	IRAM	IRAM区，设备引导使用
0xD800_0000	0xDFFF_FFFF	128MB	DMZ ROM	

S5PV210的引导区分为两个部分，分别是0x0000_0000～0x1FFF_FFFF和0xD002_0000～0xD003_FFFF的空间(空间多大？)

系统上电后，从引导区开始执行BootLoader程序。

S5PV210的SROM分为6个Bank，每个Bank大小都为128M。可支持8/16位的NOR Flash、PROM和SDRAM，并支持8/16位的数据总线（Bank0只支持16位的数据总线）。

S5PV210支持OneNAND和Flex OneNAND存储器的外部16位总线。S5PV210具有两个独立的DRAM控制器和接口，即DMC0和DMC1，分别支持8GB和4GB大小的DRAM存储器。

4.2.2 S5PV210启动流程

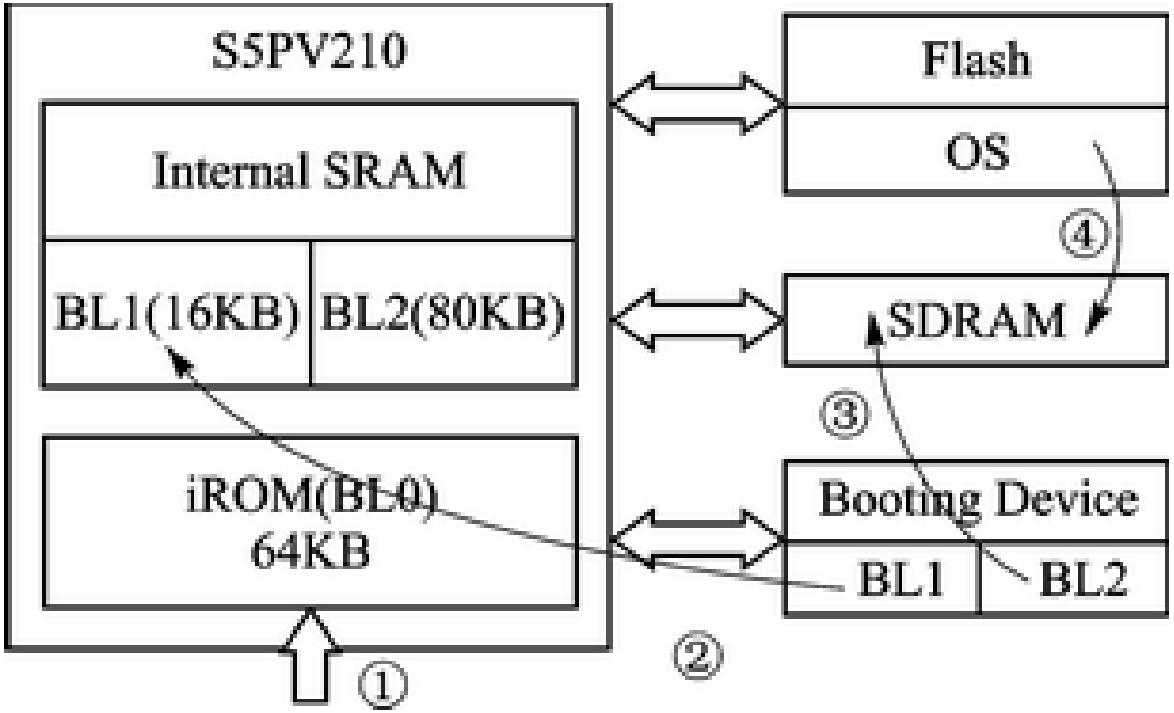


图4-3 S5PV210的启动流程

S5PV210的启动过程由BL0、BL1和BL2（BL：BootLoader）3部分代码实现，其中BL0在出厂时已经被固化到64 KB的iROM中

S5PV210上电后首先执行BL0，该段代码主要的工作序列如下所示：

（1）关看门狗时钟。

（2）初始化指令cache。

（3）初始化栈、堆。

（4）初始化块设备复制函数。

（5）初始化PLL及设置系统时钟。

（6）根据OM引脚设置，从相应启动介质复制BL1到片内SRAM的0xD002_0000地址处（其中0xD002_0010之前的16个字节存储的是BL1的校验信息和BL1的大小），并检查BL1的checksum信息，如果检查失败，IROM将自动偿试第二次启动（从SD/MMC channel 2启动）。

（7）检查是否是安全模式启动，如果是则验证BL1完整性。

（8）跳转到BL1起始地址处。

BL1的大小只有16 KB，因而一般情况下BL1负责完成的工作较少。BL1被执行后首先初始化系统时钟、内存、串口等，然后将BL2代码拷贝到Internal SRAM的BL2区中并跳转执行。从图中可以看到SRAM的BL2区的大小有80 KB，但很多情况下BL2代码的大小远远超过80 KB，所以将BL2代码拷贝到SRAM中意义不大。更好的做法是直接将BL2拷贝到容量更大的内存中，不过在拷贝之前一定要先初始化好系统时钟和内存。

BL2实际上是整个BootLoader的主体部分，因此它需要完成更多的初始化工作，例如初始化网卡、Flash等，之后BL2读取操作系统镜像到内存中运行。操作系统镜像的存放位置根据具体的开发平台而定，一般放到Flash上，也可以放到SD卡上。

4.3 Part Three

时钟系统

4.3.1 S5PV210时钟概述

CPU的系统时钟源主要是外部晶振，内部其他部分的时钟都是将外部时钟源经过一定的分频或倍频得到的。外部时钟源的频率一般不能满足系统所需要的高频条件，所以往往需要PLL（锁相环）先进行倍频处理。

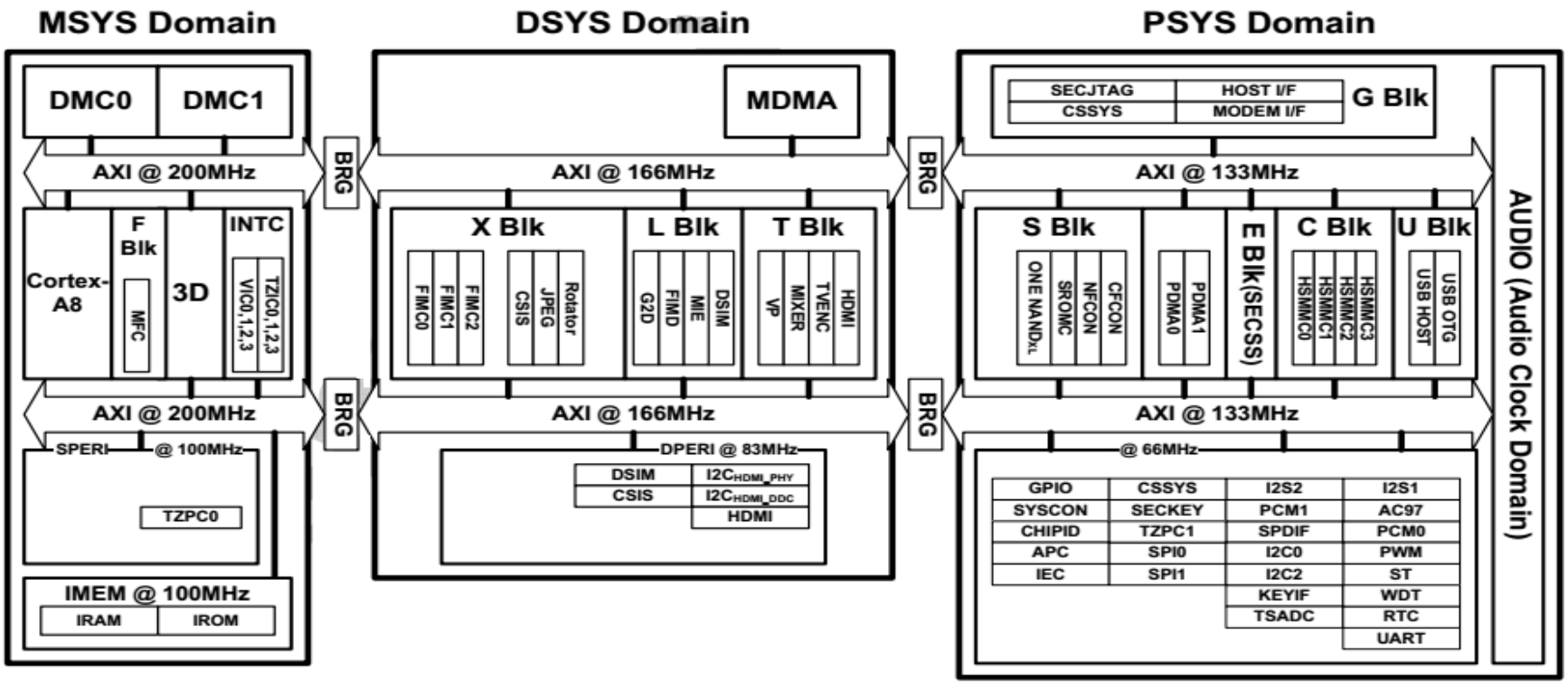


图4-4 S5PV210的时钟域

S5PV210的时钟系统包括三个时钟域(Domain)，分别是主系统时钟域 （MSYS）、显示相关的时钟域（DSYS）、外围设备的时钟域（PSYS）。

- **MSYS：**用来给CORTEX-A8处理器、DRAM控制器、3D、内部存储器（IRAM和IROM）、芯片配置界面（SPERI）、中断控制器等提供时钟。
- **DSYS：**用来给显示相关的部件提供时钟，包括FIMC、FIMD、JPEG、IPS多媒体等。
- **PSYS:**用来给外围设备（I2S、SPI、I2C、UART等）、安全子系统、低功率音频播放等提供时钟。

每个总线系统操作在200MHz（最大）、166MHz和133MHz，分别由异步总线桥梁（BRG）连接两个不同的域。

4.3.2 S5PV210的时钟结构

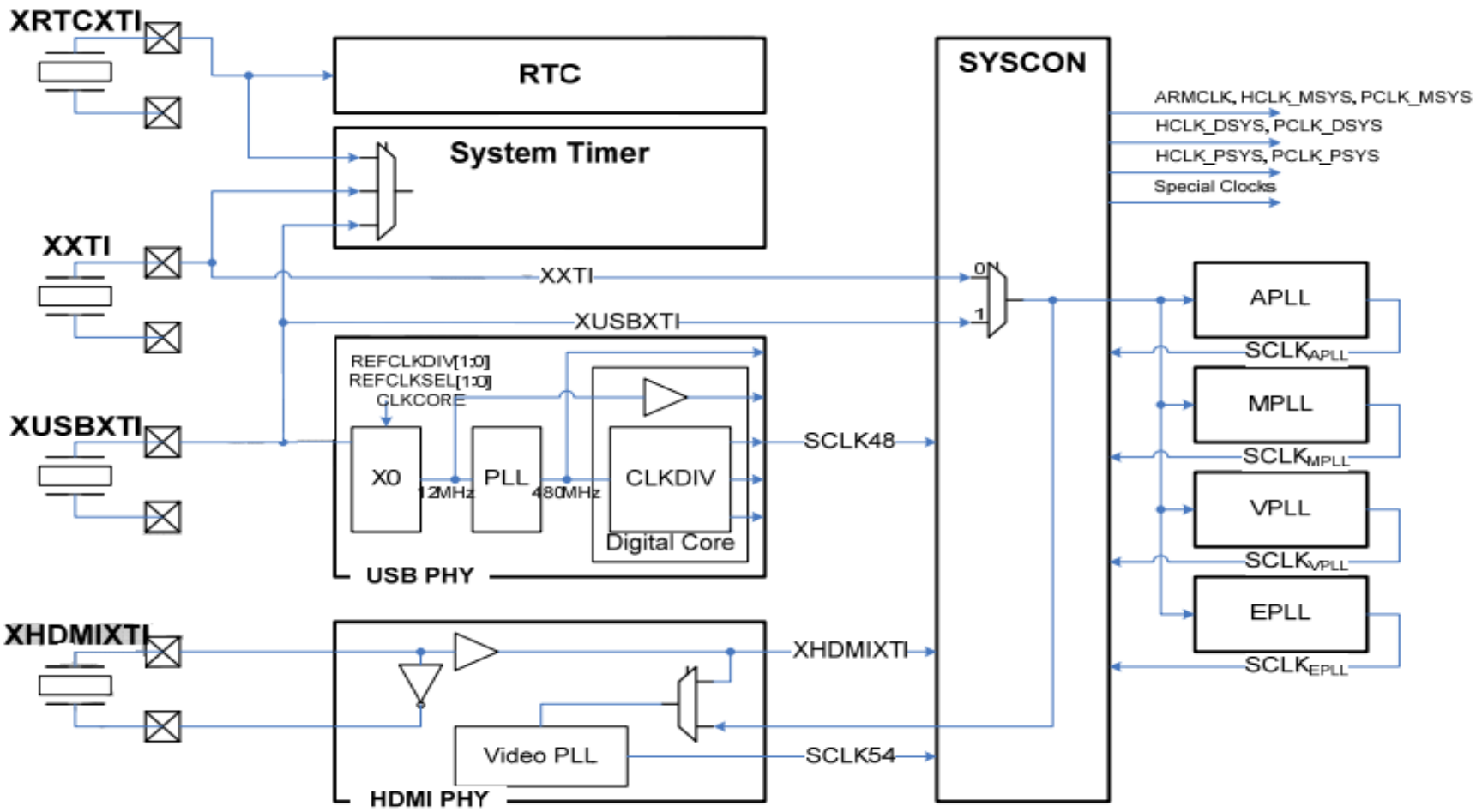


图4-5 S5PV210顶层时钟图

1. 外部时钟引脚

S5PV210的外部时钟源的时钟输入引脚有XRTCXTI、XXTI、XUSBXTI和XHDMIXTI，也可以不使用外部时钟引脚。

- XRTCXTI：将一个32.768KHz晶振提供的时钟连接到XRTCXTI和XRTCXTO引脚。RTC使用这个时钟源作为实时时钟。
- XXTI：将一个晶振提供的时钟连接到XXTI和XXTO引脚。当USB PHY没有做有效设置时，CMU和PLL使用这个时钟生成其它的时钟模块（APLL、MPLL、VPLL和EPLL）所需要的时钟。它的输入频率范围为12～50MHz。推荐使用24MHz晶振，因为IROM是基于24MHz输入时钟的。
- XUSBXTI：将一个晶振提供的时钟连接到XUSBXTI和XUSBXTO引脚，这个时钟提供给其它时钟模块（APLL、MPLL、VPLL、EPLL和USB PHY）。推荐使用24MHz晶振，因为IROM是基于24MHz输入时钟的。
- XHDMIXTI：将一个27MHz晶振提供的时钟连接到XHDMIXTI和XHDMIXTO引脚。VPLL或HDMI PHY生成54MHz时钟提供给TV编码器。

2. 时钟管理单元（CMU）

CMU使用时钟引脚（XRTCXTI、XXTI、XUSBXTI和XHDMIXTI）生成内部时钟频率、四个锁相环PLL（APLL、MPLL、EPLL和VPLL）、USB PHY和HDMI PHY。这些时钟可以选择、预分频、提供给其它扩展模块。

3. 时钟关系

3大类时钟域中，可以使用不同的分频，使其给不同部件输出所需要的时钟，各类时钟的关系如下：

- MSYS时钟域：
  - freq(ARMCLK) = freq(MOUT_MSYS)/n, where n = 1～8
  - freq(HCLK_MSYS) = freq(ARMCLK)/n, where n = 1～8
  - freq(PCLK_MSYS) = freq(HCLK_MSYS)/n, where n = 1～8
  - freq(HCLK_IMEM) = freq(HCLK_MSYS)/2
- DSYS时钟域：
  - freq(HCLK_DSYS) = freq(MOUT_DSYS)/n, where n = 1～16
  - freq(PCLK_DSYS) = freq(HCLK_DSYS)/n, where n = 1～8
- PSYS时钟域：
  - freq(HCLK_PSYS) = freq(MOUT_PSYS)/n, where n = 1～16
  - freq(PCLK_PSYS) = freq(HCLK_PSYS)/n, where n = 1～8
  - freq(SCLK_ONENAND) = freq(HCLK_PSYS)/n, where n = 1～8

- 推荐的高性能操作频率值：
  - freq(ARMCLK) = 1000 MHz
  - freq(HCLK_MSYS) = 200 MHz
  - freq(HCLK_IMEM) = 100 MHz
  - freq(PCLK_MSYS) = 100 MHz
  - freq(HCLK_DSYS) = 166 MHz
  - freq(PCLK_DSYS) = 83 MHz
  - freq(HCLK_PSYS) = 133 MHz
  - freq(PCLK_PSYS) = 66 MHz
  - freq(SCLK_ONENAND) = 133 MHz、166 MHz
- 锁相环（PLL）
  - APLL可以驱动MSYS域和DSYS域。可以产生1GHz的时钟和49:51的占空比。
  - MPLL可以驱动MSYS域和DSYS域。可以产生高达2GHz的时钟和40:60的占空比。
  - EPLL主要用于生成音频时钟。
  - VPLL主要用于生成视频系统操作时钟，54 MHz。
  - 通常，APLL驱动MSYS域、MPLL驱动DSYS域。

4.4 Part Four

GPIO接口

4.4.1 GPIO概述

GPIO（General-Purpose Input/Output Ports）全称是通用编程I/O端口。它们是CPU的引脚，可以通过它们向外输出高低电平，或者读入引脚的状态，这里的状态也是通过高电平或低电平来反应的，所以GPIO接口技术可以说是CPU众多接口技术中最为简单、常用的一种。

每个GPIO端口至少需要两个寄存器:

一个是用于控制的“通用I/O端口控制寄存器”

一个是存放数据的“通用I/O端口数据寄存器”

控制和数据寄存器的每一位和GPIO的硬件引脚相对应，由控制寄存器设置每一个引脚的数据流向，数据寄存器设置引脚输出的高低电平或读取引脚上的电平。

S5PV210共有237个GPIO端口，分成15组：

- GPA0：8输入/输出引脚。
- GPA1：4输入/输出引脚。
- GPB：8输入/输出引脚。
- GPC0：5输入/输出引脚。
- GPC1：5输入/输出引脚。
- GPD0：4输入/输出引脚。
- GPD1：6输入/输出引脚。
- GPE0、GPE1：13输入/输出引脚。
- GPF0、GPF1、GPF2、GPF3：30输入/输出引脚。
- GPG0、GPG1、GPG2、GPG3：28输入/输出引脚。
- GPH0、GPH1、GPH2、GPH3：32输入/输出引脚。
- GPP1：低功率I2S、PCM。
- GPJ0、GPJ1、GPJ2、GPJ3、GPJ4：35输入/输出引脚。
- MP0_1、MP_2、MP_3：20输入/输出引脚。
- MP0_4、MP_5、MP_6、MP_7：32输入/输出存储器引脚。

GPIO的15组引脚除了作为输入、输出引脚外，一般都还有其它功能，称为引脚复用。具体要使用引脚的哪个功能，需要通过相关的控制寄存器来设置。

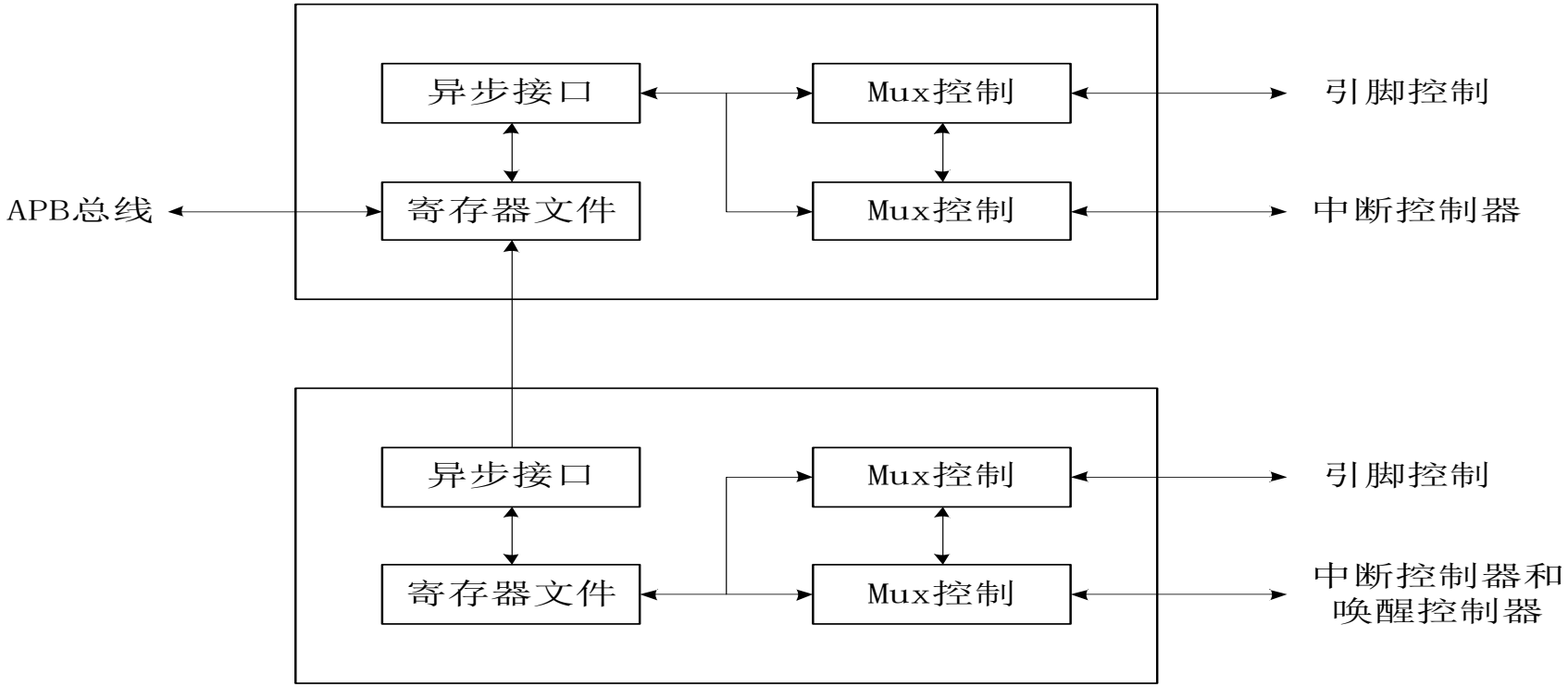


图4-7 GPIO端口功能框图

4.4.2 GPIO寄存器

每组GPIO端口都有两类控制寄存器，分别工作在正常模式和掉电模式（STOP、DEEP-STOP、睡眠模式）。

S5PV210处理器工作在正常模式下时，

正常寄存器（

如GPA0控制寄存器GPA0CON

GPA0数据寄存器GPA0DAT

GPA0上拉/下拉寄存器GPA0PUD，

GPA0驱动能力控制寄存器GPA0DRV）

工作；

进入掉电模式时，所有配置和上拉/下拉控制由掉电寄存器控制。

GPIO主要的相关寄存器：

- GPIO控制寄存器GPxnCON。用于控制GPIO的引脚功能，向该寄存器写入数据来设置相应引脚是输入/输出，还是其它功能。该寄存器中每4位控制一个引脚，**写入0000设置为输入IO口，从引脚上读入外部输入的数据；写入0001设置为输出IO口，向该位写入的数据被发送到对应的引脚上；**
- GPIO数据寄存器GPxnDAT。用于读写引脚的状态，即该端口的数据。**当引脚被设置为输出引脚，写该寄存器的对应位为1，设置该引脚输出高电平，写入0设置该引脚输出低电平；**当引脚被设置为输入引脚，读取该寄存器对应位中的数据可得到端口电平状态。
- GPIO上拉/下拉寄存器GPxnPUD，用于控制每个端口上拉/下拉电阻的使能/禁止。对应位为0时，该引脚使用上拉/下拉电阻；对应位为1时，该引脚不使用上拉/下拉电阻。
- GPIO掉电模式上拉/下拉寄存器GPxnPUDPDN，用于掉电模式下使用。每两位对应一个引脚，为00时输出0，01时输出1，10时为输入功能，11时保持原有状态。

4.4.3 GPIO操作步骤

S5PV210处理器GPIO端口操作步骤如下：

首先，确定所使用的GPIO端口的功能，如作为输入/输出引脚使用时，是否需要设置上拉/下拉电阻；作为其它功能使用时，对应S5PV210处理器的芯片手册进行设置。

其次，确定GPIO端口的输入/输出方向，通过端口设置寄存器完成端口的输入/输出功能或其它功能设置。

最后，对数据寄存器操作。

如果设置为输入引脚，读取数据寄存器对应位值，实现引脚状态的读取；

如果设置为输出引脚，通过写数据寄存器对应位值，实现引脚状态的设置。

4.4.4 一个LED灯的例子

IO口的操作是硬件控制的基础，本例子是一个最简单的IO口操作。8个LED灯分别用8个GPIO口来进行单独控制，通过I/O控制发光二极管的亮和灭。

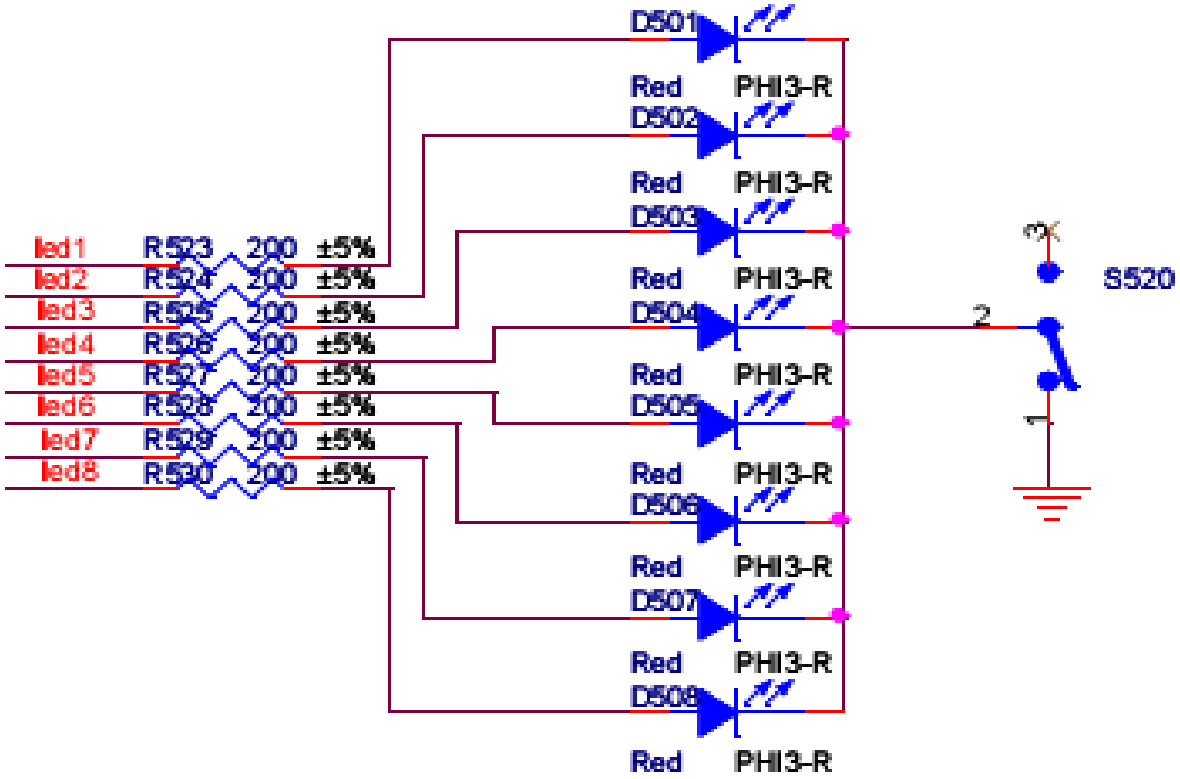


图4-8 GPIO与LED灯的连接

通过电路原理图可以得到的物理连接如下表所示：

原理图led灯标识	ARM芯片接口标识
Led1	XEINT16 /GPH2_0
Led2	XEINT17 /GPH2_1
Led3	XEINT18 /GPH2_2
Led4	XEINT19 /GPH2_3
Led5	XEINT24 /GPH3_0
Led6	XEINT25 /GPH3_1
Led7	XEINT26 /GPH3_2
Led8	XEINT27 /GPH3_3

查看GPH2相关的寄存器：

GPH2相关的寄存器有：GPH2CON, GPH2DAT, GPH2PUD以及GPH2DRV。通过对不同寄存器的操作，可以配置GPIO的功能。

表4-3　GPH2寄存器族

寄存器	物理地址	读/写属性	描述	初始值
GPH2CON	0Xe020_0C40	R/W	GPH2CON端口配置寄存器	0x00000000
GPH2DAT	0Xe020_0C44	R/W	GPH2DAT端口数据寄存器	0x00
GPH2PUD	0Xe020_0C48	R/W	GPH2PUD端口上拉寄存器	0x5555
GPH2DRV	0Xe020_0C4C	R/W	GPH2DRV端口驱动能力寄存器	0x0000

表4-4 GPH2CON端口配置寄存器

GPH2CON	位	描述	初始值
GPH2CON[0]	[3:0]	0000=Input 0001=Output 0010=Reserved 0011=KP_COL[0] 0100~1110= Reserved 1111=EXT_INT[16]	0000

表4-5 GPH2DAT端口数据寄存器

GPH2DAT	位	描述	初始值
GPH2DAT[7:0]	[7:0]	8位数据输入或者输出	0x00

表4-6 GPH2PUD端口上拉寄存器

GPH2PUD	位	描述	初始值
GPH2PUD [n]	[2n+1:2n] N=0~7	00=禁止上拉/下拉 01=下拉使能 10=上拉使能 11= Reserved	0x5555

表4-7 GPH2DRV端口驱动能力寄存器

GPH2DRV	位	描述	初始值
GPH2DRV [n]	[2n+1:2n] N=0~7	00=1x 01=2x 10=3x 11= 4x	0x00

本例通过使用**mmap**方法实现不经过内核驱动直接在用户区映射的方式来控制寄存器，从而最终控制**LED**灯的功能。

由于在应用程序中不能直接操作物理地址，因此通过**mmap**将一个文件或者其它对象映射进物理地址，应用程序就可以直接操作地址，从而达到控制寄存器的目的。在向端口写函数**port_write**中我们可以看到**mmap**方法。

```
int port_write( unsigned int n,
                unsigned int fd,
                volatile unsigned int ADDR_CON_OFFSET,
                volatile unsigned int GPIO_WR_CON,
                volatile unsigned int GPIO_WR_DAT)
{
        ADDR_START = (volatile unsigned char*)
mmap(NULL,1024*n,PROT_READ|PROT_WRITE,MAP_SHARED,fd,0xE0200000);
        if(ADDR_START == NULL)
         {
                printf("mmap err!\n");
                return -1;
        }
        *(volatile unsigned int *)(ADDR_START + ADDR_CON_OFFSET)=
        GPIO_WR_CON;
        GPIO_DAT =
         (volatile unsigned int *)(ADDR_START + ADDR_CON_OFFSET + 0x04);
        *(volatile unsigned char *)GPIO_DAT =GPIO_WR_DAT;
        return 0;
}
```

port_write中的mmap函数有如下说明：

void *mmap(void *start,size_t length,int prot,int flags,int fd,off_t offset);

- start :映射到进程空间的虚拟地址；
- length :映射空间的大小；
- prot :映射到内存的读写权限；
- flags flags:可取MAP_SHARED,MAP_PRIVATE,MAP_FIXED,如果是MAP_SHARED。此进程对映射空间的内容修改会影响到其它的进程，即对其它的进程可见，而MAP_PRIVATE，此进程修改的内容对其它的进程不可见；
- fd :要映射文件的文件标识符；
- offset: 映射文件的位置，一般从头开始。而在设备文件中，表示映射物理地址的起始地址。

程序中另外的主要调用的函数定义分别如下：

```
int open_port_device(void)
{
        int fd;
        fd = open(DEV_NAME,O_RDWR);
        if(fd<0)
        {
                printf("Open device err!\n");
                return -1;
        }
        return fd;
}
int close_port_device(int fd)
{
        close(fd);
        return 0;
}
int free_port_device(void)
{
        munmap((void *)ADDR_START,1024*16);
        return 0;
}
```

LED灯分别用8个GPIO接口进行独立控制,实现一个一个连续单独亮起，程序名称为led8，工程文件列表makefile文件内容如下：

```
CC                      = arm-linux-gcc
INSTALL                 = install
TARGET                  = led8
all : $(TARGET)
$(TARGET):led8.c led8.h
            $(CC) -static $< -o $@
clean :
            rm -rf *.o $(TARGET)
```

主程序如下：

```
//CORTEX-A8 LED TEST
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <unistd.h>
#include "led8.h"
int main(void)
{
        int fd;
        if ( (fd=open_port_device())<0 )
                exit(0);
while(1)
{
        if ( (port_write(16,fd,GPH2CON_OFFSET,0x00001111,0xff))<0)
                exit(0);
```

```
else
{
        *(volatile unsigned char *)GPIO_DAT = 0x0e;
        printf("LED 1 \n");
        sleep(1);
        *(volatile unsigned char *)GPIO_DAT = 0x0d;
        printf("LED 2 \n");
        sleep(1);
        *(volatile unsigned char *)GPIO_DAT = 0x0b;
        printf("LED 3 \n");
        sleep(1);
        *(volatile unsigned char *)GPIO_DAT = 0x07;
        printf("LED 4 \n");
        sleep(1);
        *(volatile unsigned char *)GPIO_DAT = 0x0f;
}
if( (port_write(16,fd,GPH3CON_OFFSET,0x00001111,0xff))<0)
          exit(0);
else
{
        *(volatile unsigned char *)GPIO_DAT = 0x0e;
        printf("LED 5 \n");
        sleep(1);
```

```
                *(volatile unsigned char *)GPIO_DAT = 0x0d;
                  printf("LED 6 \n");
                  sleep(1);
                  *(volatile unsigned char *)GPIO_DAT = 0x0b;
                  printf("LED 7 \n");
                  sleep(1);
                 *(volatile unsigned char *)GPIO_DAT = 0x07;
                 printf("LED 8 \n");
                 sleep(1);
                 *(volatile unsigned char *)GPIO_DAT = 0x0f;
            }
}
            free_port_device();
            close_port_device(fd);
            return 0;
}
```

除在开发板可看到LED亮灯结果外，在调试终端中显示结果如下：

```
[root@ARM /]# ./led8
LED 1
LED 2
LED 3
LED 4
LED 5
LED 6
LED 7
LED 8
LED 1
LED 2
LED 3
LED 4
LED 5
LED 6
LED 7
```

S5P4418 直流电机控制实验

```
#include "s5p4418.h"

/*********************************

 * @brief          delay_ms

 * @param[in]      int (ms)

 * @return         None

 *********************************
void delay_ms(int ms) {
    int i, j;
    while (ms--) {
        for (i = 0; i < 5; i++)
            for (j = 0; j < 514; j++)
                ;
    }
}
```

```
int main(void)
{
    GPIOA.OUTENB |= (0x1 << 20); //GPIOA20 :设置输出模式
    while (1) {
        //电机转动
        GPIOA.OUT |= 0x1 << 20;
        delay_ms(2000);
        //电机停止
        GPIOA.OUT &= ~(0x1 << 20);
        delay_ms(2000);
    }
    return 0;
}
```

S5P4418 跑马灯实验

```
**********************************

int main(void)

{

    int i = 0;

    GPIOA.OUTENB |= (0x1 << 23); //GPIOA23

    GPIOA.OUTENB |= 0xFF;                //GPIC

    //全部点亮                          //全部熄

    GPIOA.OUT &= ~(0x1 << 23);          GPIOA.

    GPIOA.OUT |= 0xFF;                  GPIOA.

    GPIOA.OUT |= 0x1 << 23;             GPIOA.

    delay_ms(500);                      delay_m

                                        while (1
```

```
    //依次点亮

    for(i = 0; i<8; i++){

        GPIOA.OUT &= ~(0x1 << 23);

        GPIOA.OUT |= 0x1 << i;

        GPIOA.OUT |= 0x1 << 23;

        delay_ms(500);

    }

    //全部熄灭

    GPIOA.OUT &= ~(0x1 << 23);

    GPIOA.OUT &= ~(0xFF);

    GPIOA.OUT |= 0x1 << 23;

    delay_ms(500);

}

return 0;
```

4.5 Part Five

串行通信接口

在数据通信中有两种常用的通信方式：串行通信和并行通信。

并行通讯是指数据的各位同时进行传送(例如数据和地址总线)，其优点是传送速度快，缺点是有多少位数据就需要多少根传输线，这在数据位数较多，传送距离较远时就不宜采用。

串行通讯是指数据一位一位地按顺序传送，其突出优点是只需一根传输线，特别适宜于远距离传输，缺点是传送速度较慢。

串行通讯中又分为异步传送和同步传送。

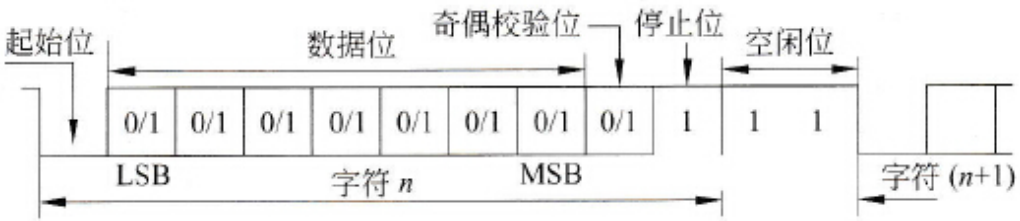
异步传送时，数据在线路上是以一个字(或称字符)为单位来传送的，各个字符之间可以是连续传送，也可以是间断传送，这完全由发送方根据需要来决定。另外，在异步传送时，发送方和接收方各用自己的时钟源来控制发送和接收。

4.5.1 串行通信方式

S5PV210处理器中采用的是异步串行通信（UART）方式。所谓异步就是指发送端和接收端使用的不是同一个时钟。异步串行通信通常以字符（或者字节）为单位组成字符帧传送。

1. 异步串行通信数据格式

异步串行通信发送的数据帧（字符帧）由4个部分组成，分别是起始位、数据位、奇偶校验位、停止位。数据帧格式如图4-10所示，



- 起始位：位于字符帧的开头，只占一位，始终为逻辑“0”低电平，表示发送端开始发送一帧数据。
- 数据位：紧跟起始位后，可取5、6、7、8位，低位在前，高位在后。
- 奇偶校验位：占一位，用于对字符传送作正确性检查。奇偶校验位是可选择的，共有三种可能，即奇偶校验、偶校验和无校验，由用户根据需要选定。
- 停止位：末尾，为逻辑“1”高电平，可取1、1.5、2位，表示一帧字符传送完毕。
- 空闲位：处于逻辑“1”高电平，表示当前线路上没有数据传输。

2. 波特率

串行通信的速率用波特率来表示，所谓波特率就是指一秒钟传送数据位的个数。每秒钟传送一个数据位就是1波特，即：1波特＝1bps（位/秒）。

在串行通信中，数据位的发送和接收分别由发送时钟脉冲和接收时钟脉冲进行定时控制。时钟频率高，则波特率高，通信速度就快；反之，时钟频率低，波特率就低，通信速度就慢。

例如每秒传送的速率为960字符/秒，而每个字符又包含10位，(1位起始位，7位数据位，1位奇偶校验位，1位停止位)，则波特率为：

960字符/秒×10位/字符=9600位/秒=9600波特

4.5.2 RS-232C串行接口

RS-232C 标准（协议）的全称是 EIA-RS-232C 标准，其中EIA (Electronic Industry Association）代表美国电子工业协会，RS（recommended standard）代表推荐标准，232是标识号，C代表RS232的最新一次修改（1969），在这之前，有RS232B、RS232A。它规定连接电缆和机械、电气特性、信号功能及传送过程。常用物理标准还有EIA-RS-422A、EIA-RS-423A、EIA-RS-485。这里只介绍EIA-RS-232C（简称232，RS232）。

1. RS-232C串行接口引脚定义

由于RS-232C标准并未定义连接器的物理特性，因此，出现了DB-25、DB-15和DB-9各种类型的连接器，其引脚的定义也各不相同。现在常用的是9针的DB-9接口，DB-9引脚顺序和定义如图4-10和表4-2所示。

图4-11 DB-9串口针脚图

针脚	信号	定义
1	DCD	载波检测
2	RXD	接收数据
3	TXD	发送数据
4	DTR	数据终端准备好
5	SGND	信号地
6	DSR	数据准备好
7	RTS	请求发送
8	CTS	清除发送
9	RI	振铃提示

表4-8 DB-9引脚定义

2. RS-232C串行接口电气特性

EIA-RS-232C对电气特性、逻辑电平和各种信号线功能都作了规定：

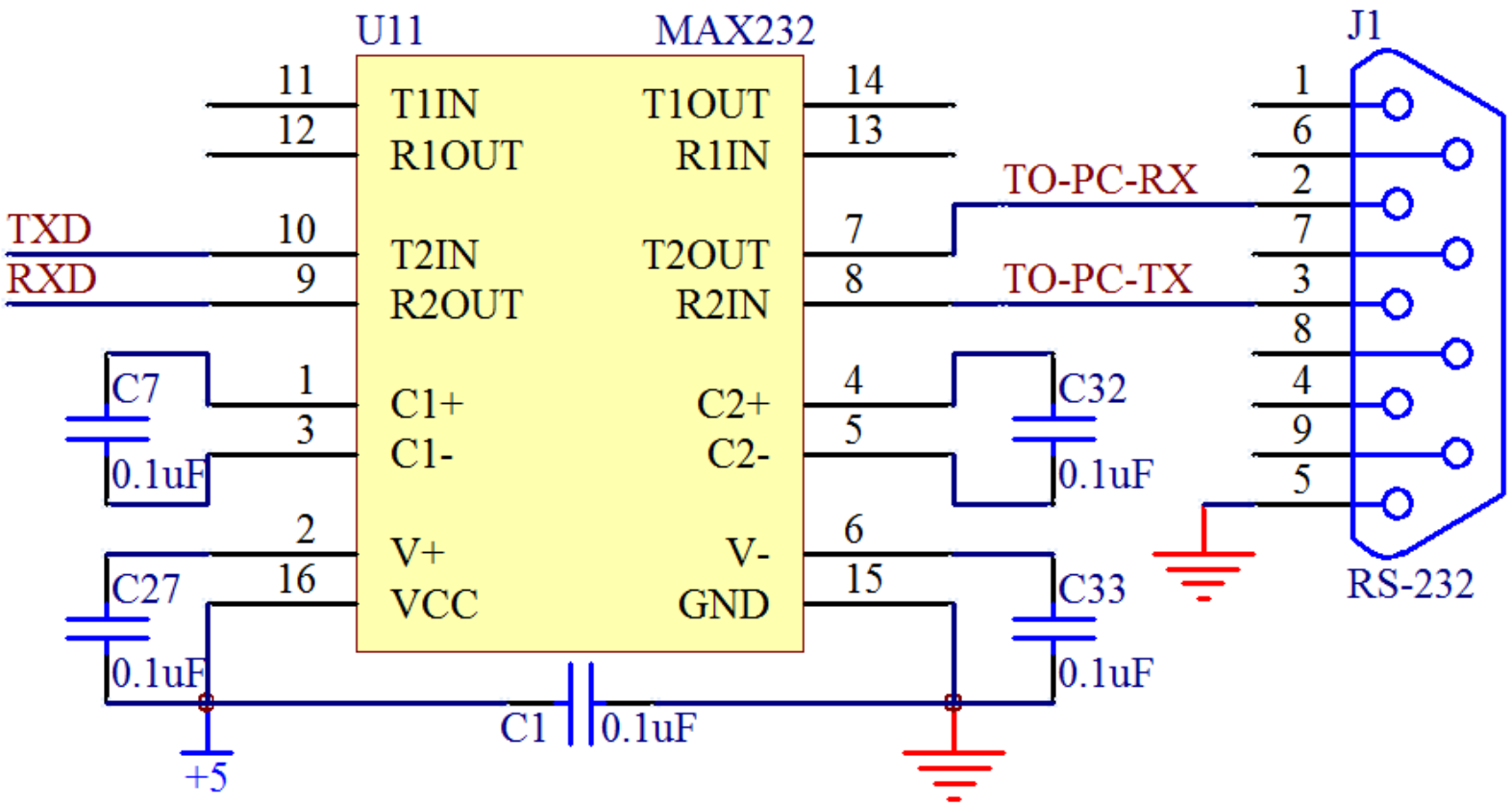
在TxD和RxD上：逻辑1 = -3V～-15V，逻辑0 = +3～+15V；

在RTS、CTS、DSR、DTR和DCD等控制线上：信号有效（接通，ON状态，正电压）= +3V～+15V；信号无效（断开，OFF状态，负电压）= -3V～-15V。

实际工作时，应保证电平在-3V～-15V或+3V～+15V之间。

EIA-RS-232C 与TTL转换：EIA RS-232C 是用正负电压来表示逻辑状态，与TTL以高低电平表示逻辑状态的规定不同。因此，为了能够同计算机接口或终端的TTL器件连接，必须在EIA-RS-232C 与TTL电路之间进行电平和逻辑关系的变换。

实现这种变换的方法可用分立元件，也可用集成电路芯片。目前较为广泛地使用集成电路转换器件，如MAX232芯片可完成TTL←→EIA双向电平转换。



4.5.3 S5PV210的异步串行通信

S5PV210处理器的UART模块提供了4个独立的异步串行输入/输出端口。每个端口都支持中断模式或DMA模式，UART可产生一个中断或发出一个DMA请求，来传输CPU和UART之间的数据。UART支持最高3Mbit/s的传输速度。每个UART通道都包含两个FIFO用来接收和发送数据，其中UART0的FIFO为256B，UART1为64B，UART2和UART3为16B。

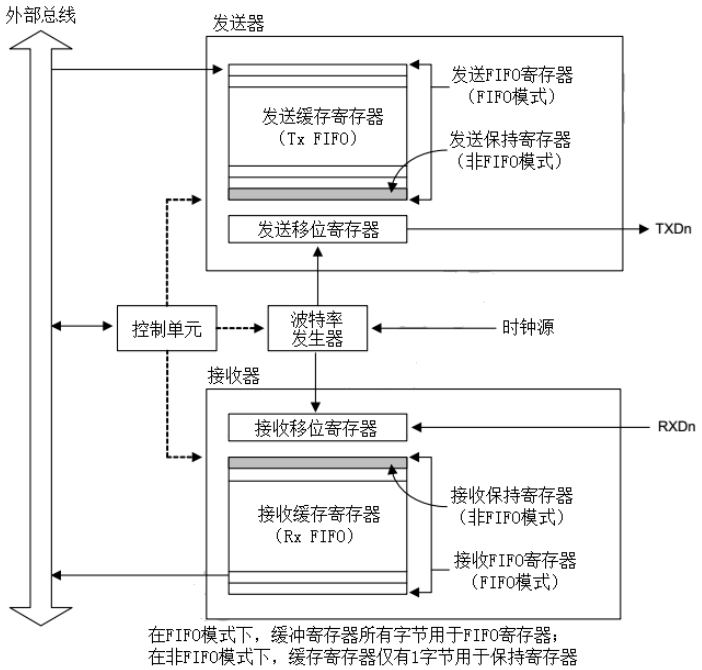


图4-12 UART结构图

S5PV210处理器每个UART包含一个波特率发生器、一个发送器、一个接收器和一个控制单元。波特率发生器使用PCLK或SCLK_UART，发送器和接收器包含FIFO和数据移位寄存器。

要发送的数据被写入Tx FIFO，然后被复制到发送移位寄存器，随后被发送引脚TxDn移出。

接收数据时，数据通过RxDn引脚移位进入接收移位寄存器中，最后被复制到Rx FIFO。

4.5.4 S5PV210的UART寄存器

S5PV210处理器的UART功能寄存器有：

- UART线控寄存器（ULCONn）。
- UART控制寄存器（UCONn）。
- UART FIFO控制寄存器（UFCONn）。
- UART MODEM控制寄存器（UMCONn）。
- UART接收发送状态寄存器（UTRSTATn）。
- UART错误状态寄存器（UERSTATn）。
- UART FIFO状态寄存器（UFSTATn）
- UART MODEM状态寄存器（UMSTATn）。
- UART发送缓存寄存器（UTXHn）。
- UART接收缓存寄存器（URXHn）。
- UART波特率除数寄存器（UBRDIVn）。

UART在程序初始化时，需要用到的寄存器有：ULCONn、UCONn、UFCONn、UMCONn、UBRDIVn等；

收发数据用到的寄存器有：UTRSTATn、UTXHn、URXHn等。

4.5.5 UART通信示例

在UART通信之前，在uart_init()函数中完成对UART的初始化，即设置UART的时钟源、传输波特率和传输数据的格式等；**在getc()和putc()函数中实现串口数据的接收和发送。**

1. 涉及的相关寄存器

2. 示例程序：

UART初始化函数uart_init()，串口接收函数getc()和串口发送函数putc()都放在uart.c文件中供main.c文件的主程序调用。

UART初始化函数uart_init()：

```
#define GPA0CON                 ( *((volatile unsigned long *)0xE0200000) )
#define GPA1CON                 ( *((volatile unsigned long *)0xE0200020) )
// UART相关寄存器地址
#define ULCON0                  ( *((volatile unsigned long *)0xE2900000) )
#define UCON0                   ( *((volatile unsigned long *)0xE2900004) )
#define UFCON0                  ( *((volatile unsigned long *)0xE2900008) )
#define UMCON0                  ( *((volatile unsigned long *)0xE290000C) )
#define UTRSTAT0                ( *((volatile unsigned long *)0xE2900010) )
#define UERSTAT0                ( *((volatile unsigned long *)0xE2900014) )
#define UFSTAT0                 ( *((volatile unsigned long *)0xE2900018) )
#define UMSTAT0                 ( *((volatile unsigned long *)0xE290001C) )
#define UTXH0                   ( *((volatile unsigned long *)0xE2900020) )
#define URXH0                   ( *((volatile unsigned long *)0xE2900024) )
#define UBRDIV0                 ( *((volatile unsigned long *)0xE2900028) )
#define UDIVSLOT0               ( *((volatile unsigned long *)0xE290002C) )
#define UINTP                   ( *((volatile unsigned long *)0xE2900030) )
#define UINTSP                  ( *((volatile unsigned long *)0xE2900034) )
#define UINTM                   ( *((volatile unsigned long *)0xE2900038) )
```

```
//波特率分频值
#define UART_UBRDIV_VAL                    35
#define UART_UDIVSLOT_VAL      0x1
//串口初始化函数
void uart_init()
{
        /**************配置引脚用于RX/TX功能*****************/
        GPA0CON = 0x22222222;
        GPA1CON = 0x2222;
        /**************设置数据格式等*****************/
        // 使能fifo
        UFCON0 = (1 << 0);
        // 无流控
        UMCON0 = 0x0;
        // 数据位:8, 无校验, 停止位: 1
        ULCON0 = (0x11 << 0);
        // 时钟：PCLK，禁止中断，使能UART发送、接收
        UCON0  = (0x01 << 2) | (0x01 << 0);
        /***************设置波特率****************/
        UBRDIV0 = UART_UBRDIV_VAL;
        UDIVSLOT0 = UART_UDIVSLOT_VAL;
}
```

```
// 串口接收一个字符
char getc(void)
{
        // 如果RX FIFO空，等待
        while (!(UTRSTAT0 & (1<<0)));     //UTRSTAT0为1,到 FIFO读数据
        // 读取数据
        return URXH0;
}
// 串口发送一个字符
void putc(char c)
{
        // 如果TX FIFO满，等待
        while (!(UTRSTAT0 & (1<<2)));   //UTRSTAT0为1,可发数据至FIFO
        // 发送数据
        UTXH0 = c;
}
```

main.c文件中，主函数main()调用函数uart_init()初始化UART串口。在while循环中，调用getc()函数接收一个字符，在将该字符的ASCII码值加1后通过调用函数putc()发送出去。

```
void uart_init(void);
int main()
{
        char c;
        // 初始化串口
        uart_init();
        while (1)
        {
                // 开发板接收字符
                c = getc();
                // 开发板发送字符c+1
                putc(c+1);
        }
        return 0;
}
```

4.6 Part Six

A/D转换器

A/D转换器是模拟信号源和CPU之间联系的接口，它的任务是将连续变化的模拟信号转换为数字信号，以便计算机和数字系统进行处理、存储、控制和显示。在工业控制和数据采集及许多其他领域中，A/D转换是不可缺少的。

根据转换原理的不同，A/D转换器可以分为以下类型：逐位比较型、积分型、计数型、并行比较型、电压一频率型等。

A/D转换器可以以独立芯片或设备形式存在，如ADC0809等；也可以在片上系统的芯片内部出现，而这一点在现在的嵌入式系统中十分常见。如基于Cortex-A8内核的处理器S5PV210具有10通道多路复用ADC，支持最大500K采样率和12位的分辨率。

A/D转换器的主要指标如下所示：

（1） 分辨率

分辨率反映A/D转换器对输入微小变化响应的能力，通常用数字输出最低位(LSB)所对应的模拟输入的电平值表示。n位A/D能反应1/2n满量程的模入电平。

(2) 精度

精度有绝对精度(Absolute Accuracy)和相对精度(Relative Accuracy)两种表示方法。

(3)转换时间

转换时间是指完成一次A/D转换所需的时间，即由发出启动转换命令信号到转换结束信号开始有效的时间间隔。

(4)电源灵敏度

电源灵敏度是指A/D转换芯片的供电电源的电压发生变化时，产生的转换误差。一般用电源电压变化1％时相当的模拟量变化的百分数来表示。

(5)量程

量程是指所能转换的模拟输入电压范围，分单极性、双极性两种类型。

(6)输出逻辑电平

多数A/D转换器的输出逻辑电平与TTL电平兼容。

(7)工作温度范围

由于温度会对比较器、运算放大器、电阻网络等产生影响，故只在一定的温度范围内才能保证额定精度指标。一般A/D转换器的工作温度范围为（0~70◦ C）。

4.7 Part Seven

本章小结

总结

本章介绍了三星公司的**S5PV210**处理器的特点和硬件结构。该处理器内部集成了丰富的常用外设模块，对其内部的**6**个主要模块的功能做了简要说明；

随后，详细介绍了**S5PV210**处理器的地址空间分配以及虚拟地址和物理地址的映射，并简要介绍了处理器的启动流程，对处理器的时钟系统构成进行了解析；

最后详细介绍了最常用的外设：**GPIO**和**UART**。

通过这两种接口的介绍，初步掌握嵌入式处理器外设的控制、驱动方法，在今后学习到其它接口外设时能够举一反三。